Introduction

In a monolithic application, we usually use one database for everything. This single database handles both read and write operations. That means that the database is both working for **complex join queries**, and also perform CUD (Create, Update, Delete) operations. But, when the application gets more complicated, handling these read and write tasks can become really hard to manage.

CQRS stands for **Command Query Responsibility Segregation**. It's a design pattern that splits the operations of a system into two distinct parts: **commands** (which modify data i.e., writes) and **queries** (which read data i.e., reads). This separation allows for more efficient handling and scaling of each operation.

Why Use CQRS in Microservices?

1. **Scalability:** Separate scaling of read and write workloads. You can scale your query and command services independently based on demand.
2. **Optimized Performance:** Each service can be optimized for its specific task (read or write), leading to improved performance.
3. **Simplified Complexity:** By separating concerns, it becomes easier to manage the complexity of each microservice.
4. **Flexibility in Technology Choice:** Different technology stacks can be used for the read side and the write side if it suits the specific needs better.